

what is spark structured streaming, Spark Structured Streaming Architecture , Stateless VS Stateful Transformations , PySpark Streaming Query Tutorial , Triggers in Apache Spark Streaming , Output Modes in Apache Spark Streaming , ForEachBatch in PySpark Streaming , Event Time VS Processing Time , Tumbling Window, Sliding Window, and Sessions ,Watermarks in Apache Spark Streaming

explain me in detailed all these for apache spark to master the basics of spark streaming
and at the end write in one cell small project using spark structured streaming

Absolutely. These are the **core concepts you should understand to master the basics of Spark Structured Streaming**. I'll build them from the ground up and connect them together.

1. What is Spark Structured Streaming?

Spark Structured Streaming is Spark's engine for processing **continuously arriving data**.

Instead of waiting until all data exists:

```
graph TD; A[CSV files] --> B[Batch Spark]; B --> C[Process all data]; C --> D[Result];
```

Upgrade

Streaming works like:

```
graph TD; A[Data arrives continuously] --> B[ ];
```

Spark Structured Streaming
↓
Process new data
↓
Update results continuously

For example, imagine an application producing:

```
user_id=101, event=login
user_id=102, event=purchase
user_id=103, event=login
user_id=101, event=logout
...
```

Spark can process these events as they arrive.

Common streaming sources

Spark Structured Streaming can read from things such as:

- Kafka
- Files
- Rate source
- Socket source
- Other supported streaming sources

A very common Big Data architecture is:

```
Application
↓
Kafka
↓
Spark Structured Streaming
↓
Transform / Aggregate
↓
Parquet / Delta / Database / Kafka
```

For your learning path, the important combination is:

```
Kafka → Spark Structured Streaming → Storage
```

2. Why is it called "Structured" Streaming?

Traditional Spark Streaming was based around lower-level concepts.

Structured Streaming is built on the **Spark SQL/DataFrame API**.

So you can use familiar operations:

 **Python**



 **Run**

```
df.select(...)
df.filter(...)
df.groupBy(...)
df.withColumn(...)
df.join(...)
```

The important idea is:

Structured Streaming treats a stream of data like an ever-growing table.

Imagine events arriving:

Time 1:

user	event
101	login
102	login

Time 2:

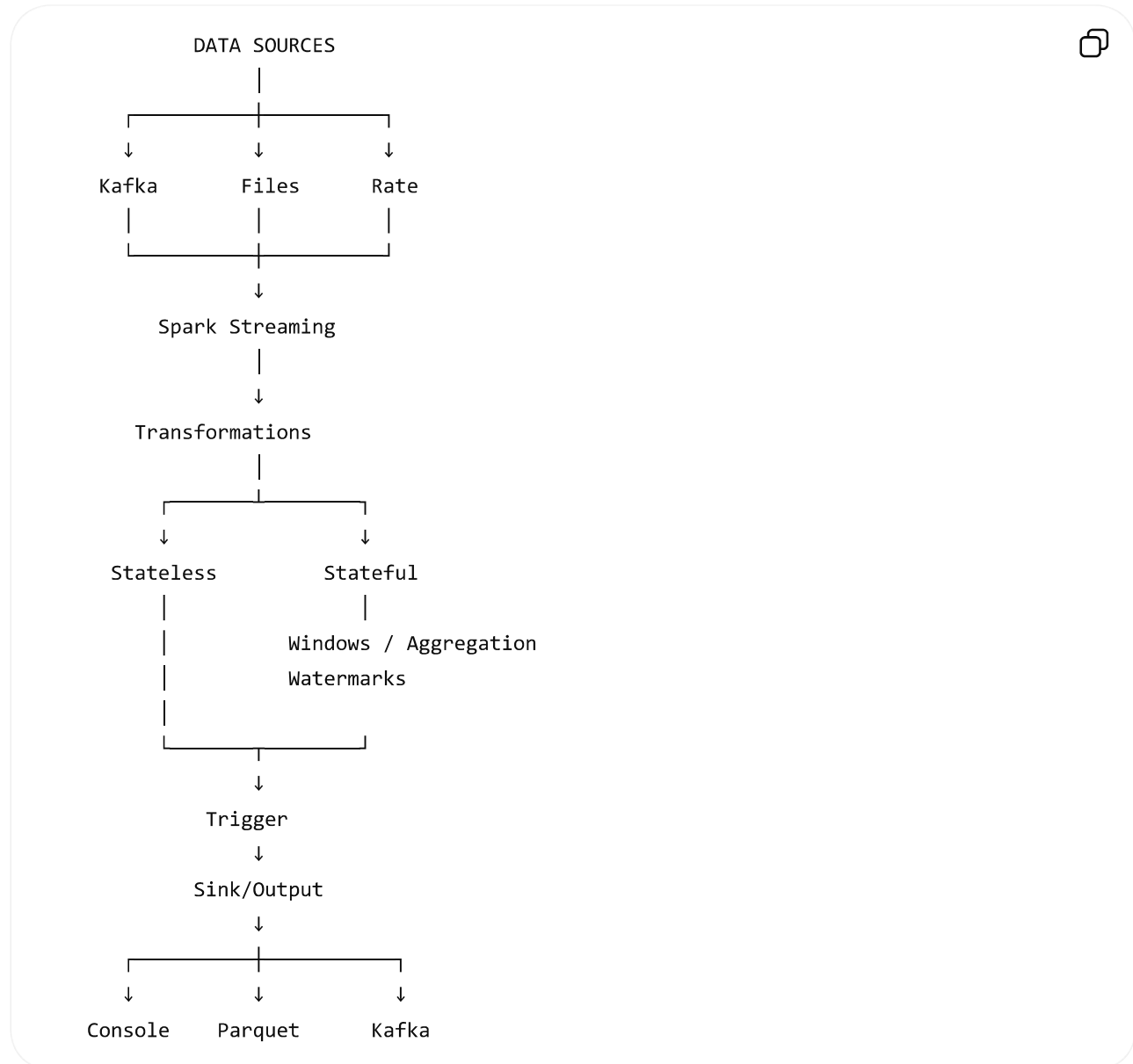
user	event
101	login
102	login
103	purchase
101	logout

Conceptually, Spark sees the **streaming data** as an **unbounded table**.

You don't normally wait for this table to finish because it never finishes.

3. Spark Structured Streaming Architecture

A simplified architecture is:



Let's understand each component.

4. Streaming Source

The source is where streaming data comes from.

Example:

`</>` Python

```
df = (  
    spark.readStream  
        .format("rate")
```



Run


```
.option("rowsPerSecond", 5)
.load()

)
```

This creates a streaming DataFrame.

Another important example is Kafka:

</> Python



▶ Run

```
df = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("subscribe", "events")
        .load()
)
```

The result is a streaming DataFrame.

5. Streaming DataFrame

A streaming DataFrame looks similar to a normal DataFrame:

</> Python



▶ Run

```
df.select("value")
```

But there is an important difference.

Batch:

</> Python



▶ Run

```
df = spark.read.csv(...)
```

means:

Read the data that currently exists.

Streaming:

</> Python



▶ Run

```
df = spark.readStream.csv(...)
```

means:

Keep watching for new data.

So:

```
spark.read
```



→ batch

```
spark.readStream
```



→ streaming

6. Stateless vs Stateful Transformations

This is one of the **most important concepts** in Structured Streaming.

Stateless transformation

A stateless operation processes each piece of data **without remembering previous data**.

Examples:

</> Python



▶ Run

```
select()  
filter()  
withColumn()  
drop()
```

Example:

</> Python



▶ Run

```
df.filter(df["age"] > 18)
```

Spark doesn't need to remember previous rows.

If:

Incoming:



Alice 20

Bob 15

John 30

The result is:

Alice 20



John 30

Each row can be processed independently.

Think:

Input → Process → Output



No memory of previous data is required.

7. Stateful Transformations

A stateful transformation needs information from previous data.

Examples include:

</> Python



▶ Run

```
groupBy()  
count()  
sum()  
avg()  
window()  
dropDuplicates()
```

For example:

</> Python



▶ Run

```
df.groupBy("user_id").count()
```

Spark needs to remember:

user 101 → 5 events
user 102 → 8 events
user 103 → 2 events



When a new event arrives:

user 101



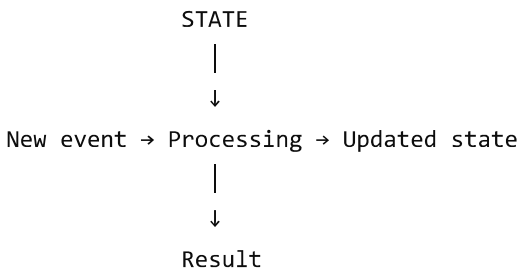
Spark updates:

user 101 → 6 events



So Spark maintains **state**.

Conceptually:



8. Why Stateful Streaming is More Difficult

State can become very large.

Imagine:

1 million users
×
years of events



Spark can't simply forget everything.

Therefore Spark needs mechanisms such as:

- State Store
- Checkpointing
- Watermarks

These are extremely important for production streaming.

9. Streaming Query

A streaming DataFrame by itself doesn't start processing.

For example:

</> Python



▶ Run

```
df = spark.readStream.format("rate").load()
```

Nothing is continuously running yet.

You need to create a **streaming query**.

Example:

</> Python



▶ Run

```
query = (  
    df.writeStream  
        .format("console")  
        .start()  
)  
  
query.awaitTermination()
```

The basic pattern is:

```
readStream  
  ↓  
transform  
  ↓  
writeStream  
  ↓  
start()  
  ↓  
awaitTermination()
```



10. Basic PySpark Streaming Query

Here is a very simple example:

 Python



 Run

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("StreamingExample")
    .master("local[*]")
    .getOrCreate()
)

df = (
    spark.readStream
    .format("rate")
    .option("rowsPerSecond", 5)
    .load()
)

result = df.select("timestamp", "value")

query = (
    result.writeStream
    .format("console")
    .outputMode("append")
    .start()
)

query.awaitTermination()
```

The rate source generates data continuously.

For example:

Batch: 0

timestamp value
2026-08-26 12:00:01 0
2026-08-26 12:00:01 1
...

Batch: 1

timestamp value

```
2026-08-26 12:00:02      5
2026-08-26 12:00:02      6
...
```

11. Triggers

A trigger determines when Spark should process the available data.

Think:

"When should Spark execute another micro-batch?"

For example:

`</>` Python



▶ Run

```
.trigger(processingTime="10 seconds")
```

means:

```
Process
↓
wait/process every 10 seconds
↓
Process
↓
wait 10 seconds
↓
Process
```



12. Important Trigger Types

Processing time

`</>` Python

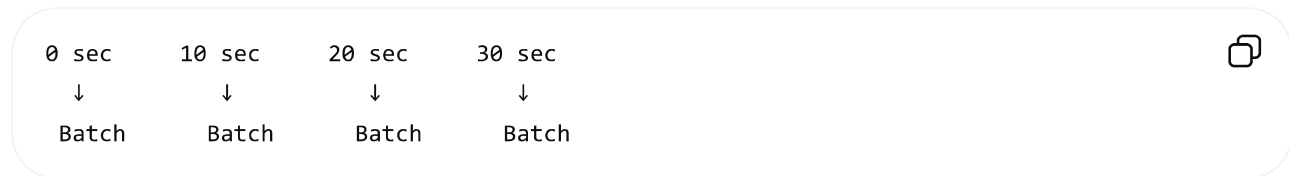


▶ Run

```
.trigger(processingTime="10 seconds")
```

Spark processes data according to a time interval.

Conceptually:



AvailableNow

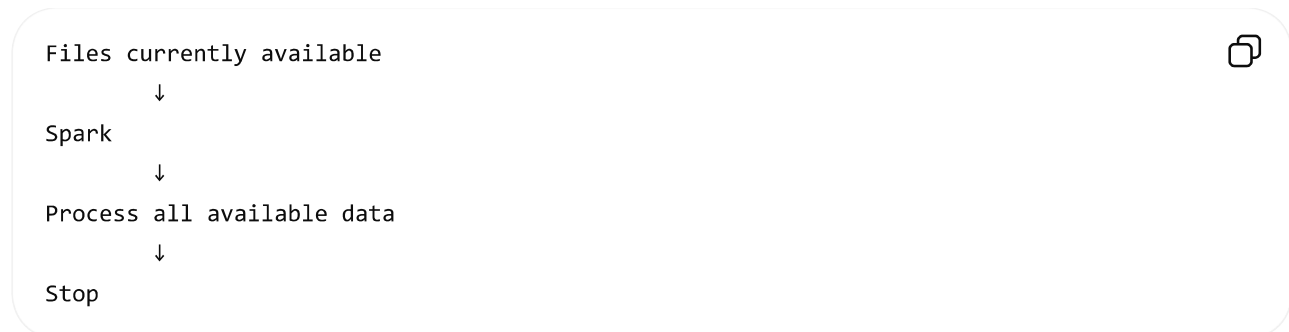
```
</> Python Run  
  
.trigger(availableNow=True)
```

Spark processes the data that is currently available and then terminates.

This is extremely useful when you want:

Process everything available right now, then stop.

Example:



Once

You may also encounter:

```
</> Python Run  
  
.trigger(once=True)
```

It processes available data once and then stops.

For modern Spark applications, **AvailableNow** is generally more useful for incremental processing.

13. Output Modes

Output mode determines:

What should Spark write to the sink?

There are three important modes:

Append
Complete
Update



14. Append Mode

</> Python



Run

```
.outputMode("append")
```

Only **new rows** are written to the output.

Example:

First batch:

Alice
Bob



Output:

Alice
Bob



Next batch:

Charlie



Output:

Charlie



The old rows aren't rewritten.

Good for

Stateless transformations such as:

</> Python



▶ Run

```
filter()
select()
withColumn()
```

Example:

</> Python



▶ Run

```
df.filter(df.value > 10)
```

15. Complete Mode

</> Python



▶ Run

```
.outputMode("complete")
```

Spark outputs the **entire result table** every time.

Example aggregation:

</> Python



▶ Run

```
df.groupBy("user_id").count()
```

Result:

```
101 → 5
102 → 8
103 → 2
```



Next batch:

```
101 → 6  
102 → 8  
103 → 3
```



Complete mode outputs the entire table again.

This is mainly useful for aggregations.

16. Update Mode

</> Python



▶ Run

```
.outputMode("update")
```

Only rows whose results have changed are output.

Example:

Initial state:

```
101 → 5  
102 → 8  
103 → 2
```



New event for user 101:

```
101 → 6
```



Update mode can output:

```
101 → 6
```



instead of the entire table.

17. Output Mode Summary

Mode	What is written?
Append	Only new final rows
Update	Rows whose values changed
Complete	Entire result table

Think:

Append → NEW
Update → CHANGED
Complete → EVERYTHING



18. Event Time vs Processing Time

This is one of the most important streaming concepts.

Suppose an event is generated at:

10:00:00



but reaches Spark at:

10:00:05



We have two timestamps.

Event time

When the event actually happened.

```
event_time = 10:00:00
```



Processing time

When Spark processed the event.

```
processing_time = 10:00:05
```



19. Why Event Time Matters

Imagine users are sending events from mobile phones.

A user loses internet connection.

The event occurs:

10:00



But the device reconnects at:

10:10



Spark receives the event at:

10:10



If you use processing time:

10:10



But the event really belongs to:

10:00



Therefore event-time processing is extremely important when dealing with delayed or out-of-order data.

20. Windowing

Windowing means:

Divide an infinite stream into finite time periods.

Instead of:

Infinite stream



we create:

```
Window 1 | Window 2 | Window 3
-----|-----|-----
    0-10   | 10-20   | 20-30
```



This allows us to ask questions like:

How many purchases happened every 10 minutes?

or:

What is the average temperature during each 5-minute period?

21. Tumbling Window

A **tumbling window** has:

- fixed duration
- no overlap

Example:

```
Window size = 10 minutes
```



You get:

```
00 — 10
10 — 20
20 — 30
30 — 40
```



Each event belongs to one window.

Example

Events:

```
10:01
10:03
10:08
10:12
```



10:18

10:25

Windows:

10:00 - 10:10

10:01

10:03

10:08

10:10 - 10:20

10:12

10:18

10:20 - 10:30

10:25



In PySpark:

</> Python



Run

```
from pyspark.sql.functions import window

result = (
    df
    .groupBy(
        window("timestamp", "10 minutes")
    )
    .count()
)
```

22. Sliding Window

A sliding window has:

- window duration
- slide duration
- overlapping windows

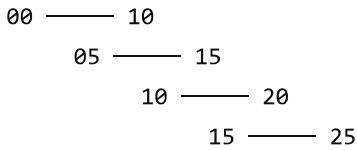
Example:

Window = 10 minutes

Slide = 5 minutes



You get:



```

00 ——— 10
    05 ——— 15
        10 ——— 20
            15 ——— 25
  
```

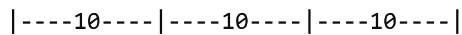


Windows overlap.

An event can therefore belong to multiple windows.

23. Tumbling vs Sliding

Tumbling



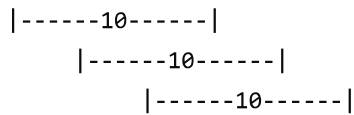
```

|----10----|----10----|----10----|
  
```



No overlap.

Sliding



```

|-----10-----|
  |-----10-----|
    |-----10-----|
  
```



Overlap.

Example use cases

Tumbling:

Number of orders every 10 minutes.

Sliding:

Number of orders during the last 10 minutes, calculated every 5 minutes.

24. Session Windows

Session windows are different.

They are based on **activity/inactivity**, not fixed clock intervals.

Suppose a user performs:

```
10:00 login
10:02 view product
10:05 add cart
```



Then no activity for a while.

If the session timeout is:

```
10 minutes
```



the session might end after 10 minutes of inactivity.

Then:

```
User 101
```



```
10:00 — 10:02 — 10:05
                ↓
            inactivity
                ↓
        session ends
```

If the user comes back later:

```
10:30 purchase
```



that could create another session.

Session windows are useful for:

- website sessions
- user activity
- clickstreams
- browsing behavior

25. Window Types Summary

Window	Overlap?	Based on
Tumbling	No	Fixed interval
Sliding	Yes	Fixed interval + slide
Session	No fixed interval	Activity/inactivity

Think:

Tumbling → fixed + no overlap

Sliding → fixed + overlap

Session → activity-based



26. Watermarks

Watermarks are one of the most important concepts in streaming.

The problem is **late data**.

Imagine:

Event A → event time 10:01

Event B → event time 10:02

Event C → event time 10:03



Everything is fine.

But later:

Event D → event time 09:55



Event D arrives late.

Spark needs to know:

How long should I wait for late events?

That's where **watermarks** help.

27. Example of Watermark

Suppose:

 Python



 Run

```
df.withWatermark("timestamp", "10 minutes")
```

This tells Spark:

I expect events to arrive up to approximately 10 minutes late according to the event-time column.

Conceptually:

Event time:



10:00

10:05

10:10

10:15

10:20

↑

watermark

Spark can eventually decide that sufficiently old windows are unlikely to receive more data.

28. Why Watermarks Are Important

Without watermarks, stateful operations can potentially keep state for a very long time.

Imagine:

User 1

User 2

User 3

...

User 10 million



Spark needs to maintain state.

Watermarks allow Spark to eventually remove state associated with old windows when the watermark has passed them, subject to the operation's semantics.

So:

```
graph TD; A[Watermark] --> B[Control late data]; B --> C[Limit old state]; C --> D[Manage memory];
```

29. Watermark Example

Suppose:

Python



Run

```
from pyspark.sql.functions import window

result = (
    df
    .withWatermark("timestamp", "10 minutes")
    .groupBy(
        window("timestamp", "5 minutes")
    )
    .count()
)
```

Here:

```
graph TD; A[timestamp] --> B[Watermark: 10 minutes]; B --> C[5-minute windows]; C --> D[Aggregation];
```

The watermark is based on **event time**, not Spark's processing clock.

30. Watermark Does NOT Mean "Wait Exactly 10 Minutes"

This is a common beginner misunderstanding.

If you write:

</> Python



Run

```
.withWatermark("timestamp", "10 minutes")
```

it does **not** simply mean:

Wait exactly 10 minutes before processing every event.

Instead, it provides Spark with a threshold for handling **late event-time data** and managing state.

31. ForEachBatch

`foreachBatch()` is very useful.

Normally:

Streaming DataFrame



`writeStream`



Sink



But `foreachBatch()` allows you to run custom code for **each micro-batch**.

Example:

</> Python



Run

```
def process_batch(batch_df, batch_id):  
    print("Batch:", batch_id)  
  
    batch_df.show()  
  
query = (  
    df.writeStream  
        .foreachBatch(process_batch)
```

```
.start()  
)
```

Spark calls:

```
process_batch(batch_df, batch_id)
```



for every micro-batch.

32. Why ForEachBatch is Useful

You can use it to:

- write to databases
- run custom transformations
- use batch APIs
- perform upserts
- write to multiple destinations
- call custom Python logic

Conceptually:

```
Streaming Data  
  ↓  
Micro-batch  
  ↓  
foreachBatch()  
  ↓  
Custom processing  
  ↓  
Database / files / API / etc.
```



33. ForEachBatch Example

Suppose Spark receives:

```
Batch 0:  
Alice  
Bob
```

```
Batch 1:
```



Charlie
David

Spark executes:

</> Python



▶ Run

```
process_batch(batch0, 0)
```

then:

</> Python



▶ Run

```
process_batch(batch1, 1)
```

So:

```
Micro-batch 0 → function  
Micro-batch 1 → function  
Micro-batch 2 → function  
...
```



34. Checkpointing

Checkpointing is critical for reliable streaming.

You should understand:

Checkpointing stores information that Spark needs to recover a streaming query.

For example:

</> Python



▶ Run

```
query = (  
    df.writeStream  
        .format("parquet")  
        .option("checkpointLocation", "/tmp/checkpoint")  
        .start("/tmp/output")  
)
```

A checkpoint directory can contain information related to:

```
checkpoint/  
├─ commits/  
├─ offsets/  
├─ sources/  
├─ state/  
└─ metadata
```



The exact contents depend on the query and Spark version.

35. Why Checkpointing?

Suppose your streaming application crashes:

```
Kafka  
↓  
Spark  
↓  
CRASH
```



After restarting, Spark needs information about what it had already processed.

Checkpointing helps Spark recover the streaming query's progress/state.

Think:

```
Streaming Query  
↓  
Checkpoint  
↓  
Recovery information
```

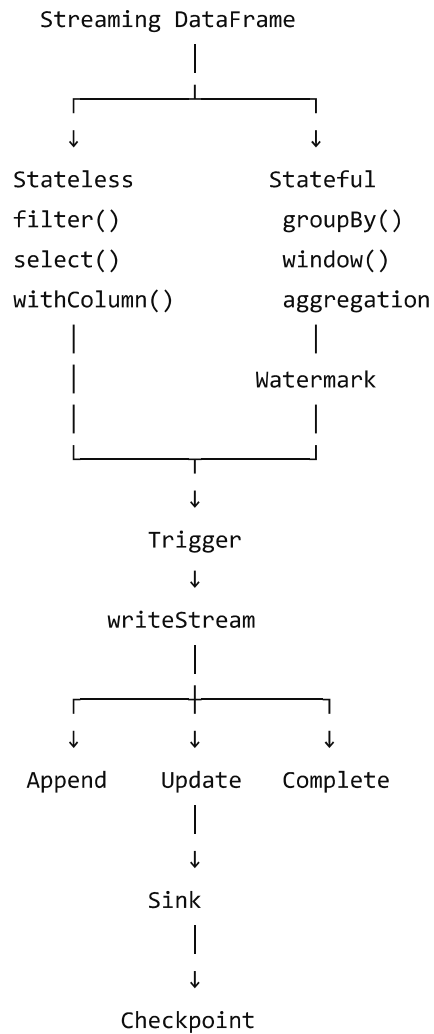


36. Complete Mental Model

At this point, connect everything:

```
SOURCE  
|  
↓  
readStream  
|  
↓
```





37. The Most Important Things to Remember

If you're preparing for interviews, remember these definitions.

Structured Streaming

A scalable stream-processing engine built on Spark's structured APIs that processes continuously arriving data using DataFrames/Datasets.

Stateless

Processing that doesn't need to remember previous records.

Stateful

Processing that maintains information from previous records.

Event time

When the event actually happened.

Processing time

When Spark processes the event.

Tumbling window

Fixed-size, non-overlapping time windows.

Sliding window

Fixed-size overlapping time windows.

Session window

Windows based on periods of activity separated by inactivity.

Watermark

A mechanism for handling late event-time data and allowing Spark to clean up old state.

Trigger

Determines when Spark processes available streaming data.

Output mode

Determines what results are written to the sink.

foreachBatch

Allows custom processing of each micro-batch as a normal batch DataFrame.

Checkpoint

Stores progress/state information needed for fault tolerance and recovery.

38. Small Project: File Streaming → Transformation → Window → Parquet

Since you're learning Spark Structured Streaming before integrating Kafka, a good first project is:

JSON files

↓

Spark Structured Streaming

↓

Parse JSON

↓

Event time

↓

5-minute window

↓

Count events

↓

Parquet

↓

Checkpoint



Put some JSON files into:

stream_input/



For example:

</> JSON



```
{"user_id":101,"event":"login","timestamp":"2026-08-26T12:00:01"}
{"user_id":102,"event":"purchase","timestamp":"2026-08-26T12:01:20"}
{"user_id":101,"event":"logout","timestamp":"2026-08-26T12:02:10"}
```

Then Spark continuously watches the directory for new JSON files.

Here is the **small complete project in one cell**:

</> Python



▶ Run

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

```
# =====
```

```

# 1. Create SparkSession
# =====

spark = (
    SparkSession.builder
        .appName("StructuredStreamingProject")
        .master("local[*]")
        .getOrCreate()
)

spark.sparkContext.setLogLevel("WARN")

# =====
# 2. Define JSON schema
# =====

schema = StructType([
    StructField("user_id", IntegerType(), True),
    StructField("event", StringType(), True),
    StructField("timestamp", StringType(), True)
])

# =====
# 3. Read streaming JSON files
# =====

df = (
    spark.readStream
        .schema(schema)
        .json("stream_input")
)

# =====
# 4. Convert timestamp string to timestamp type
# =====

df = df.withColumn(
    "event_time",
    to_timestamp("timestamp")
)

# =====
# 5. Add watermark
#     Accept events that arrive up to 10 minutes late
# =====

```

```

df = df.withWatermark(
    "event_time",
    "10 minutes"
)

# =====
# 6. Create a 5-minute tumbling window
#   Count events by event type
# =====

result = (
    df.groupBy(
        window("event_time", "5 minutes"),
        "event"
    )
    .count()
)

# =====
# 7. Write results to Parquet
# =====

query = (
    result.writeStream
        .format("parquet")
        .outputMode("append")
        .option("path", "stream_output")
        .option("checkpointLocation", "stream_checkpoint")
        .trigger(processingTime="10 seconds")
        .start()
)

# =====
# 8. Keep the streaming query running
# =====

query.awaitTermination()

```

This small project demonstrates almost everything you've just learned:

```

readStream
  ↓
Schema
  ↓

```



Event time
↓
Watermark
↓
Stateful aggregation
↓
Tumbling window
↓
Trigger
↓
Append output mode
↓
Parquet sink
↓
Checkpoint

Once you understand this project, the next major step is to replace:

JSON files



with:

Kafka



giving you the very important real-world architecture:

Producer
↓
Kafka Topic
↓
Spark Structured Streaming
↓
Parse Kafka messages
↓
Event Time
↓
Watermark
↓
Window / Aggregation
↓
Parquet / Database / Kafka



That is the natural progression from **basic Spark Streaming** → **Kafka + Spark Streaming projects**.



Is this conversation helpful so far?   

ChatGPT can make mistakes. Check important info.